# Casos de prueba de integración

> Generado desde `scripts/casos_de_prueba.py` (fuente única). Para registrar resultados usa `casos_de_prueba.xlsx` (columnas *Estado*, *Resultado real*, *Evidencia*).

**Total:** 104 casos — 49 automatizados (pytest), 16 semi-automáticos, 39 manuales.

**Tipo:** A = automatizado · S = semi-automático (script/simulador + revisión) · M = manual.
**Prioridad:** Alta = bloquea la aceptación; Media = debe cumplirse; Baja = deseable.

## Resumen por nivel

| Nivel | Componente | Casos | Alta | A | S | M |
|---|---|---|---|---|---|---|
| N0 | Entorno AWS y Docker | 8 | 7 | 0 | 3 | 5 |
| N1 | Sensor ↔ ESP32 | 10 | 4 | 0 | 0 | 10 |
| N2 | ESP32 ↔ Red WiFi / NTP | 8 | 3 | 0 | 0 | 8 |
| N2 | ESP32 ↔ API (contrato HTTP) | 32 | 14 | 28 | 0 | 4 |
| N3 | API ↔ Base de datos | 18 | 11 | 12 | 5 | 1 |
| N4 | Extremo a extremo | 8 | 4 | 0 | 3 | 5 |
| N5 | No funcionales | 9 | 4 | 2 | 3 | 4 |
| N6 | Seguridad básica | 11 | 5 | 7 | 2 | 2 |

## Requisitos y trazabilidad

| Requisito | Descripción | Casos |
|---|---|---|
| RF-01 | El sistema mide temperatura y humedad con el sensor conectado al ESP32. | IT-SEN-01, IT-SEN-03, IT-SEN-04, IT-SEN-05, IT-SEN-06 |
| RF-02 | El ESP32 se conecta a la red WiFi y recupera la conexión tras una caída. | IT-NET-01, IT-NET-02, IT-NET-03, IT-NET-08 |
| RF-03 | El ESP32 envía cada lectura a la API en JSON, autenticado con API key. | IT-ENV-02, IT-API-18, IT-API-22, IT-DB-17 |
| RF-04 | La API valida el contenido y persiste las lecturas en InfluxDB (series de tiempo) y la configuración en PostgreSQL. | IT-ENV-04, IT-API-01, IT-API-05, IT-API-07, IT-API-08, IT-API-10, IT-API-24, IT-API-28, IT-DB-01, IT-DB-05, IT-DB-06, IT-DB-07, IT-DB-10, IT-DB-14, IT-DB-15, IT-E2E-02 |
| RF-05 | No se pierden ni se duplican lecturas ante reintentos, cortes o reinicios. | IT-SEN-02, IT-SEN-09, IT-API-02, IT-API-12, IT-API-19, IT-API-20, IT-DB-02, IT-DB-03, IT-DB-16, IT-E2E-01, IT-E2E-03, IT-E2E-04, IT-NFR-02, IT-NFR-08 |
| RF-06 | Las lecturas pueden consultarse por cliente, sensor y rango de tiempo. | IT-API-14, IT-API-15, IT-API-32, IT-E2E-06 |
| RF-07 | Se pueden agregar nuevas magnitudes (p. ej. PM2.5) sin cambiar el esquema. | IT-DB-04, IT-E2E-07 |

| Requisito | Descripción | Casos |
|---|---|---|
| RF-08 | Un dashboard de Grafana muestra las lecturas almacenadas (temperatura, humedad, estado de los dispositivos). | IT-ENV-07, IT-E2E-08 |
| RF-09 | El dispositivo se configura sin recompilar mediante un portal cautivo (WiFi, servidor, cliente, sensor, ubicación, ciclo). | IT-NET-06, IT-NET-07 |
| RF-10 | Clientes y sensores se registran en PostgreSQL; la API rechaza lecturas de clientes no registrados y auto-enrola sensores. | IT-API-23, IT-API-26, IT-API-27, IT-API-30, IT-API-31 |
| RF-11 | El sistema mide la calidad del aire (MQ135) además de temperatura y humedad (DHT11). | IT-SEN-07, IT-SEN-08, IT-SEN-10, IT-API-29 |
| RNF-01 | Latencia de entrega (ESP32 → BD) con p95 menor a 2 s en condiciones normales. | IT-NFR-01, IT-NFR-03, IT-NFR-04 |
| RNF-02 | El sistema se recupera solo tras caídas de red, API, BD o energía. | IT-ENV-01, IT-ENV-05, IT-ENV-06, IT-ENV-08, IT-NET-05, IT-API-17, IT-API-21, IT-DB-08, IT-DB-11, IT-DB-12, IT-DB-13, IT-NFR-05, IT-NFR-06, IT-NFR-07, IT-NFR-09 |
| RNF-03 | Seguridad básica: autenticación, validación de entradas, bases no expuestas, token de Grafana de solo lectura, sin secretos en el código. | IT-ENV-03, IT-API-03, IT-API-04, IT-API-06, IT-API-09, IT-API-13, IT-API-16, IT-DB-18, IT-SEC-01, IT-SEC-02, IT-SEC-03, IT-SEC-04, IT-SEC-05, IT-SEC-06, IT-SEC-07, IT-SEC-08, IT-SEC-09, IT-SEC-10, IT-SEC-11 |
| RNF-04 | Trazabilidad temporal: cada lectura tiene hora del dispositivo (NTP) y hora del servidor. | IT-NET-04, IT-API-11, IT-API-25, IT-DB-09, IT-E2E-05 |

## N0 — Entorno AWS y Docker

### IT-ENV-01 — Acceso SSH a la EC2 y versión de Python

- **Requisito:** RNF-02 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** Laboratorio iniciado; EC2 en ejecución.
- **Pasos:** 1. ssh -i labsuser.pem ec2-user@ 2. python3 --version
- **Datos:** IP pública / Elastic IP
- **Resultado esperado:** Acceso correcto; Python ≥ 3.11 disponible.

### IT-ENV-02 — Puerto de la API alcanzable desde la red del ESP32

- **Requisito:** RF-03 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** SG `sg-api` con regla de entrada TCP 8000; API en ejecución.
- **Pasos:** 1. Desde un celular con datos móviles y desde el WiFi del ESP32: abrir http://:8000/health
- **Datos:** URL /health
- **Resultado esperado:** Responde 200 con {"status":"ok","db":"ok"} desde ambas redes.

**IT-ENV-03 — Stack docker compose sano; InfluxDB y PostgreSQL sin puertos publicados**

- **Requisito:** RNF-03 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** EC2 con Docker; `.env` creado.
- **Pasos:** 1. En la EC2: docker compose up -d --build 2. docker compose ps (todos healthy) 3. docker compose exec postgres psql -U iot -d iot -c 'SELECT 1' 4. docker compose exec influxdb influx ping 5. docker compose port postgres 5432 y docker compose port influxdb 8086
- **Datos:** docker-compose.yml
- **Resultado esperado:** api, influxdb y postgres en estado healthy; `SELECT 1` devuelve 1; `influx ping` responde OK; ningún puerto publicado para 5432 ni 8086.

**IT-ENV-04 — Esquema de PostgreSQL y tokens de InfluxDB**

- **Requisito:** RF-04 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** Contenedores iniciados con volúmenes vacíos (aplican `db/init.sql` y `db/influx-init.sh`).
- **Pasos:** 1. docker compose exec postgres psql -U iot -d iot -c '\dt' 2. Verificar clientes, sensores, alertas y lecturas_vistas (clave primaria sensor_id+seq) 3. docker compose exec influxdb influx auth list: el token de la API tiene lectura+escritura y el de Grafana solo lectura
- **Datos:** db/init.sql · db/influx-init.sh
- **Resultado esperado:** Existen dispositivo, lectura y medicion con índices y restricciones.

**IT-ENV-05 — Contenedores con política de reinicio**

- **Requisito:** RNF-02 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** Stack en ejecución.
- **Pasos:** 1. docker inspect -f '{{.Name}} {{.HostConfig.RestartPolicy.Name}}' $(docker compose ps -q) 2. curl localhost:8000/health
- **Datos:** —
- **Resultado esperado:** Los cuatro contenedores usan `unless-stopped`; /health = 200 con postgres e influxdb ok.

**IT-ENV-06 — Prueba de humo del entorno**

- **Requisito:** RNF-02 · **Prioridad:** Alta · **Tipo:** Semi-automático
- **Precondiciones:** Stack docker compose operativo.
- **Pasos:** 1. En la EC2: API_URL=http://localhost:8000 API_KEY=… GRAFANA_URL=http://localhost:3000 ./scripts/smoke_env.sh
- **Datos:** smoke_env.sh
- **Resultado esperado:** Todas las verificaciones [OK], código de salida 0.
- **Automatización:** `scripts/smoke_env.sh`

**IT-ENV-07 — Grafana con datasource y dashboard aprovisionados**

- **Requisito:** RF-08 · **Prioridad:** Alta · **Tipo:** Semi-automático
- **Precondiciones:** Stack en ejecución; lecturas enviadas con el simulador.
- **Pasos:** 1. Abrir http://:3000 e iniciar sesión con el usuario de .env 2. Connections → Data sources → InfluxDB → Save & test 3. Abrir el dashboard «NextGuard · Monitoreo IoT» y elegir cliente y sensor
- **Datos:** grafana/
- **Resultado esperado:** El datasource responde «datasource is working»; los paneles de temperatura, humedad y calidad del aire muestran las lecturas del simulador.

**IT-ENV-08 — Persistencia de datos tras bajar y subir el stack**

- **Requisito:** RNF-02 · **Prioridad:** Alta · **Tipo:** Semi-automático
- **Precondiciones:** Lecturas cargadas en la BD.
- **Pasos:** 1. docker compose down (sin -v) 2. docker compose up -d 3. GET /api/v1/lecturas?cliente=...&sensor=... y abrir el dashboard
- **Datos:** —
- **Resultado esperado:** Las lecturas y el dashboard siguen disponibles; la API reconecta sola a ambas bases.

## N1 — Sensor ↔ ESP32

**IT-SEN-01 — Lectura DHT válida y dentro de rango**

- **Requisito:** RF-01 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** USE_DHT=1; DHT cableado (VCC, GND, DATA con pull-up); monitor serie a 115200.
- **Pasos:** 1. Cargar firmware 2. Observar 10 muestras en el monitor serie 3. Comparar con un termómetro/higrómetro de referencia
- **Datos:** DHT11 o DHT22
- **Resultado esperado:** T entre -40 y 80 °C y H entre 0 y 100 %; diferencia con la referencia dentro de la tolerancia del sensor (DHT22 ±0,5 °C; DHT11 ±2 °C).

**IT-SEN-02 — Lectura inválida (NaN) no se envía y el sistema sigue**

- **Requisito:** RF-05 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** Firmware ejecutándose con DHT real.
- **Pasos:** 1. Desconectar el pin DATA durante 30 s 2. Reconectarlo
- **Datos:** —
- **Resultado esperado:** Se imprime '[SENSOR] lectura inválida' y no se encola nada; al reconectar se retoman las lecturas sin reiniciar.

**IT-SEN-03 — Frecuencia de muestreo**

- **Requisito:** RF-01 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** Firmware ejecutándose.
- **Pasos:** 1. Registrar durante 5 min las marcas de tiempo de [MUESTRA] 2. Calcular el periodo medio
- **Datos:** INTERVALO_MS = 10000
- **Resultado esperado:** Periodo medio 10 s ±5 %; sin ráfagas ni huecos.

**IT-SEN-04 — Respuesta a un cambio físico**

- **Requisito:** RF-01 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** Firmware ejecutándose y llegando datos a la BD.
- **Pasos:** 1. Exhalar sobre el sensor 5 s 2. Consultar las últimas lecturas por API
- **Datos:** —
- **Resultado esperado:** La humedad sube de forma visible y el cambio aparece en la BD.

**IT-SEN-05 — Modo simulado sin hardware**

- **Requisito:** RF-01 · **Prioridad:** Baja · **Tipo:** Manual
- **Precondiciones:** USE_DHT=0.
- **Pasos:** 1. Cargar firmware 2. Observar 20 muestras
- **Datos:** —
- **Resultado esperado:** Valores plausibles (T ≈ 22 °C variando ≤ 0,3 °C por muestra; H entre 0 y 100 %).

**IT-SEN-06 — DHT11 (módulo HW-036) con lecturas coherentes**

- **Requisito:** RF-01 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** USE_DHT=1, DHT_TYPE=DHT11; DATA en GPIO4, alimentación 3,3 V; módulo verificado (S, +, −).
- **Pasos:** 1. Cargar firmware 2. Observar 10 muestras 3. Comparar con un termómetro/higrómetro de referencia
- **Datos:** DHT11
- **Resultado esperado:** T y H dentro de rango y a ±2 °C / ±5 %HR de la referencia; sin NaN sostenidos.

**IT-SEN-07 — MQ135: salida analógica nunca supera 3,3 V en el GPIO**

- **Requisito:** RF-11 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** Divisor resistivo (p. ej. 10 kΩ / 20 kΩ) entre AO y GPIO34; módulo alimentado a 5 V; GND común.
- **Pasos:** 1. ANTES de conectar al GPIO, medir con multímetro la tensión tras el divisor en aire limpio y con una fuente de alcohol cerca 2. Conectar AO al GPIO34 3. Cargar

firmware con USE_MQ135=1
- **Datos:** —
- **Resultado esperado:** La tensión tras el divisor es siempre ≤ 3,3 V; `calidad_aire_mv` entre 0 y 3300.

**IT-SEN-08 — MQ135: precalentamiento y lectura estable**

- **Requisito:** RF-11 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** USE_MQ135=1; MQ135 alimentado.
- **Pasos:** 1. Encender y observar el monitor serie 5 min 2. Dejar 10 min más en aire limpio
- **Datos:** MQ135_PRECALENTAMIENTO_MS = 180000
- **Resultado esperado:** Durante los primeros 3 min solo se envían T y H; después se envían `co2_equiv` (ppm) y `calidad_aire_raw`; variación < 5 % en aire limpio estable.

**IT-SEN-09 — Falla del DHT no bloquea la lectura del MQ135**

- **Requisito:** RF-05 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** DHT11 y MQ135 en funcionamiento.
- **Pasos:** 1. Desconectar DATA del DHT durante 30 s 2. Observar el monitor y la API 3. Reconectar
- **Datos:** —
- **Resultado esperado:** Las muestras siguen enviándose con `calidad_aire_*`; al reconectar vuelven T y H sin reiniciar.

**IT-SEN-10 — Gas simulado sin hardware**

- **Requisito:** RF-11 · **Prioridad:** Baja · **Tipo:** Manual
- **Precondiciones:** USE_MQ135=2.
- **Pasos:** 1. Cargar firmware 2. Observar 20 muestras
- **Datos:** —
- **Resultado esperado:** `co2_equiv` entre 400 y 2000 ppm con variación ≤ 25 ppm por muestra.

## N2 — ESP32 ↔ Red WiFi / NTP

**IT-NET-01 — Conexión WiFi inicial e IP**

- **Requisito:** RF-02 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** Red 2.4 GHz con WPA2; config.h correcto.
- **Pasos:** 1. Encender el ESP32 2. Observar el monitor serie
- **Datos:** SSID/clave
- **Resultado esperado:** Se imprime la IP asignada en menos de 20 s.
- **Automatización:** `wifi_diagnostico.ino`

**IT-NET-02 — Clave WiFi incorrecta: motivo registrado y sin bloqueo**

- **Requisito:** RF-02 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** config.h con clave errónea.
- **Pasos:** 1. Cargar firmware 2. Observar 2 min
- **Datos:** Clave inválida
- **Resultado esperado:** Se registra el motivo de desconexión; el muestreo continúa y las muestras se encolan (sin cuelgue ni reinicio).

**IT-NET-03 — Caída del router durante la ejecución**

- **Requisito:** RF-02 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** ESP32 enviando con normalidad.
- **Pasos:** 1. Apagar el router 2 min 2. Encenderlo 3. Esperar 3 min
- **Datos:** —
- **Resultado esperado:** El ESP32 se reconecta solo; las muestras acumuladas se entregan (sin faltas ni duplicados).

**IT-NET-04 — Hora NTP y campo ts_dispositivo**

- **Requisito:** RNF-04 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** Internet disponible.
- **Pasos:** 1. Arrancar y revisar '[MUESTRA] … hora=NTP' 2. Consultar la API y ver ts_dispositivo 3. Repetir con Internet bloqueado en el router
- **Datos:** —
- **Resultado esperado:** Con NTP: ts_dispositivo en UTC correcto. Sin NTP: ts_dispositivo = null y la lectura se guarda igual.

**IT-NET-05 — Entrega según intensidad de señal**

- **Requisito:** RNF-02 · **Prioridad:** Baja · **Tipo:** Manual
- **Precondiciones:** Dispositivo alimentado por batería/USB estable.
- **Pasos:** 1. Medir 10 min a tres distancias (RSSI ≈ -50, -70, -80 dBm) 2. Comparar enviadas vs guardadas
- **Datos:** RSSI
- **Resultado esperado:** Entrega ≥ 99 % con RSSI ≥ -80 dBm.

**IT-NET-06 — Portal cautivo: primer encendido guarda los campos en flash**

- **Requisito:** RF-09 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** ESP32 sin configuración guardada (o BOOT 3 s al encender).
- **Pasos:** 1. Conectar el celular a la red SENSOR-XXXX 2. Abrir http://192.168.4.1 3. Completar WiFi, servidor, API key, cliente, sensor, ubicación y ciclo 4. Guardar y esperar el reinicio

- **Datos:** Cliente cliente-001, sensor temp-lab-01, ciclo 30 s
- **Resultado esperado:** El ESP32 reinicia conectado a la red indicada; el monitor muestra sensor, cliente, ciclo y servidor guardados; tras otro reinicio no vuelve a abrir el portal.

#### IT-NET-07 — Restablecer la configuración con BOOT

- **Requisito:** RF-09 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** ESP32 configurado.
- **Pasos:** 1. Mantener BOOT 3 s al encender 2. Observar el monitor serie
- **Datos:** —
- **Resultado esperado:** Se imprime '[CFG] BOOT mantenido: se borran WiFi y configuración' y se abre el portal.

#### IT-NET-08 — Tres fallos de WiFi abren el portal

- **Requisito:** RF-02 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** ESP32 configurado.
- **Pasos:** 1. Cambiar la clave o apagar el router ~2 min
- **Datos:** —
- **Resultado esperado:** Tras 3 intentos de 30 s se abre el portal SENSOR-XXXX; al guardar una red válida reanuda el envío.

## N2 — ESP32 ↔ API (contrato HTTP)

#### IT-API-01 — Crear lectura válida → 201

- **Requisito:** RF-04 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** API con API key válida.
- **Pasos:** 1. POST /api/v1/lecturas con payload válido
- **Datos:** payload estándar
- **Resultado esperado:** 201 y {estado:guardado, duplicado:false}.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_01_crear_lectura_201`

#### IT-API-02 — Reintento con el mismo seq → 200 idempotente

- **Requisito:** RF-05 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** Lectura seq=7 ya guardada.
- **Pasos:** 1. Reenviar la misma lectura
- **Datos:** seq=7
- **Resultado esperado:** 200 con {duplicado:true}; el GET devuelve 1 sola lectura.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_02_duplicado_es_idempotente_200`

**IT-API-03 — Sin API key → 401**

- **Requisito:** RNF-03 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST sin cabecera X-API-Key
- **Datos:** —
- **Resultado esperado:** 401 y no se guarda nada.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_03_sin_api_key_401`

**IT-API-04 — API key incorrecta → 401**

- **Requisito:** RNF-03 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con X-API-Key errónea
- **Datos:** —
- **Resultado esperado:** 401.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_04_api_key_incorrecta_401`

**IT-API-05 — Lectura sin medición → 422**

- **Requisito:** RF-04 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST sin tipo/valor ni mediciones, y con mediciones=[]
- **Datos:** —
- **Resultado esperado:** 422.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_05_sin_medicion_422`

**IT-API-06 — sensor con caracteres inválidos → 422**

- **Requisito:** RNF-03 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con sensor='sensor 01;'
- **Datos:** —
- **Resultado esperado:** 422.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_06_sensor_invalido_422`

**IT-API-07 — Valor NaN → 422 (no 500)**

- **Requisito:** RF-04 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con "valor": NaN (JSON no estándar)
- **Datos:** —
- **Resultado esperado:** 422 con detalle sin el valor recibido; la API no falla con 500.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_07_valor_nan_422`

**IT-API-08 — JSON malformado → 422**

- **Requisito:** RF-04 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con cuerpo '{no es json'
- **Datos:** —
- **Resultado esperado:** 422.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_08_json_malformado_422`

**IT-API-09 — Payload demasiado grande → 413**

- **Requisito:** RNF-03 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** MAX_BODY_BYTES configurado.
- **Pasos:** 1. POST con cuerpo mayor al límite
- **Datos:** ≥ 5 KB en pruebas
- **Resultado esperado:** 413.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_09_payload_grande_413`

**IT-API-10 — Campo desconocido → 422**

- **Requisito:** RF-04 · **Prioridad:** Baja · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con un campo extra
- **Datos:** —
- **Resultado esperado:** 422 (esquema estricto).
- **Automatización:** `tests/test_api_contrato.py::test_it_api_10_campo_extra_rechazado_422`

**IT-API-11 — Sin timestamp se acepta**

- **Requisito:** RNF-04 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST sin timestamp 2. GET de la lectura
- **Datos:** —
- **Resultado esperado:** 201; ts_dispositivo = null.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_11_timestamp_ausente_permitido`

**IT-API-12 — Lote con nuevas y duplicadas**

- **Requisito:** RF-05 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** seq=1 ya guardado.
- **Pasos:** 1. POST /batch con seq 1, 2, 3
- **Datos:** 3 lecturas
- **Resultado esperado:** {insertadas:2, duplicadas:1}.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_12_batch_mezcla_nuevas_y_duplicadas`

**IT-API-13 — Lote de más de 100 → rechazado**

- **Requisito:** RNF-03 · **Prioridad:** Baja · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST /batch con 101 lecturas
- **Datos:** 101 lecturas
- **Resultado esperado:** 413 o 422.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_13_batch_mas_de_100_422`

**IT-API-14 — GET filtra por sensor y limit**

- **Requisito:** RF-06 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** Lecturas de 2 sensores.
- **Pasos:** 1. GET ?sensor=… 2. GET ?limit=2
- **Datos:** —
- **Resultado esperado:** Cantidades exactas por filtro.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_14_get_filtra_por_sensor_y_limit`

**IT-API-15 — GET devuelve mediciones completas**

- **Requisito:** RF-06 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** Lectura guardada.
- **Pasos:** 1. GET ?sensor=…
- **Datos:** —
- **Resultado esperado:** Tipos, valores, rssi y seq idénticos a lo enviado.
- **Automatización:**
  `tests/test_api_contrato.py::test_it_api_15_get_devuelve_mediciones_completas`

**IT-API-16 — GET requiere API key**

- **Requisito:** RNF-03 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. GET sin X-API-Key
- **Datos:** —
- **Resultado esperado:** 401.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_16_get_requiere_api_key`

**IT-API-17 — /health sin clave y refleja ambas bases**

- **Requisito:** RNF-02 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. GET /health
- **Datos:** —
- **Resultado esperado:** 200 {status:ok, postgres:ok, influxdb:ok}.

- **Automatización:** `tests/test_api_contrato.py::test_it_api_17_health_no_requiere_clave`

**IT-API-18 — ESP32 real: primer envío correcto**

- **Requisito:** RF-03 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** API accesible; portal cautivo configurado con servidor, API key y cliente correctos.
- **Pasos:** 1. Encender el ESP32 2. Esperar la primera muestra 3. Ver el monitor serie
- **Datos:** —
- **Resultado esperado:** Se imprime '[API] 201 OK' y la lectura aparece en GET /api/v1/lecturas.

**IT-API-19 — ESP32 con API key errónea: 401, backoff y sin pérdida**

- **Requisito:** RF-05 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** Portal con una API key incorrecta.
- **Pasos:** 1. Encender 3 min 2. Corregir la API key desde el portal (BOOT 3 s al encender)
- **Datos:** —
- **Resultado esperado:** Se registra 'fallo 401' con espera creciente (2, 4, 8… hasta 60 s); las muestras se conservan en la cola mientras no se llene.

**IT-API-20 — API caída 3 min: la cola se vacía al recuperar**

- **Requisito:** RF-05 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** ESP32 enviando con normalidad.
- **Pasos:** 1. docker compose stop api (3 min) 2. docker compose start api 3. Comparar seq del monitor serie vs InfluxDB
- **Datos:** —
- **Resultado esperado:** Sin faltas ni duplicados; se usan envíos por lote para vaciar la cola; el orden se conserva.

**IT-API-21 — Timeout de red: el ESP32 no se bloquea**

- **Requisito:** RNF-02 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** ESP32 enviando con normalidad.
- **Pasos:** 1. Quitar la regla del SG que permite el puerto 8000 (los paquetes se descartan) 2. Observar 2 min 3. Restaurar la regla
- **Datos:** —
- **Resultado esperado:** Cada intento termina por timeout (≈5 s), el muestreo sigue a su ritmo y al restaurar se entrega todo.

**IT-API-22 — Formato de la clase {sensor, cliente, ubicacion, tipo, valor, unidad} → 201**

- **Requisito:** RF-03 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** Cliente registrado.

- **Pasos:** 1. POST con el formato de la clase
- **Datos:** temperatura 22,5 °C
- **Resultado esperado:** 201 {estado:guardado}; el GET devuelve la medición.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_22_formato_de_la_clase_201`

**IT-API-23 — Cliente no registrado → 403**

- **Requisito:** RF-10 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con un cliente inexistente
- **Datos:** —
- **Resultado esperado:** 403 y no se guarda nada.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_23_cliente_no_registrado_403`

**IT-API-24 — Valor fuera del rango físico → 422**

- **Requisito:** RF-04 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con temperatura 250 °C y humedad 101 %
- **Datos:** 250 °C
- **Resultado esperado:** 422; lo rechazado nunca llega a InfluxDB.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_24_fuera_de_rango_fisico_422`

**IT-API-25 — Timestamp en el futuro → 422**

- **Requisito:** RNF-04 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con timestamp 2999-01-01
- **Datos:** —
- **Resultado esperado:** 422.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_25_timestamp_en_el_futuro_422`

**IT-API-26 — El sensor se auto-enrola con su primera lectura**

- **Requisito:** RF-10 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** Cliente registrado.
- **Pasos:** 1. POST dos veces la misma lectura de un sensor nuevo 2. GET /clientes//sensores
- **Datos:** —
- **Resultado esperado:** sensor_nuevo=true y luego false; el sensor aparece con su última lectura.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_26_sensor_se_auto_enrola_con_su_primera_lectura`

**IT-API-27 — Sin auto-enrolamiento un sensor desconocido → 403**

- **Requisito:** RF-10 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** AUTO_ENROLAR_SENSORES=false.
- **Pasos:** 1. POST de un sensor no registrado
- **Datos:** —
- **Resultado esperado:** 403.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_27_sin_auto_enrolamiento_sensor_desconocido_403`

**IT-API-28 — (tipo, valor) y mediciones a la vez → 422**

- **Requisito:** RF-04 · **Prioridad:** Baja · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con ambas formas 2. POST con tipo sin valor
- **Datos:** —
- **Resultado esperado:** 422.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_28_simple_y_mediciones_a_la_vez_422`

**IT-API-29 — Rango del MQ135 en ppm (300 a 10000)**

- **Requisito:** RF-11 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST co2_equiv=520 2. POST co2_equiv=10001
- **Datos:** —
- **Resultado esperado:** 201 y 422 respectivamente.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_29_rango_del_mq135_en_ppm`

**IT-API-30 — Clientes: crear, repetir (409) y listar**

- **Requisito:** RF-10 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST /clientes 2. POST /clientes con el mismo id 3. GET /clientes y /clientes/
- **Datos:** —
- **Resultado esperado:** 201, 409 y listado con el cliente y su cantidad de sensores.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_30_clientes_crear_y_repetir_409`

**IT-API-31 — Cliente inexistente → 404**

- **Requisito:** RF-10 · **Prioridad:** Baja · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. GET /clientes/nadie y /clientes/nadie/sensores
- **Datos:** —
- **Resultado esperado:** 404.

- **Automatización:** `tests/test_api_contrato.py::test_it_api_31_cliente_inexistente_404`

#### IT-API-32 — Lecturas de un sensor por cliente

- **Requisito:** RF-06 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** 3 lecturas guardadas.
- **Pasos:** 1. GET /clientes//sensores//lecturas?limite=2
- **Datos:** —
- **Resultado esperado:** 200 con 2 lecturas.
- **Automatización:** `tests/test_api_contrato.py::test_it_api_32_lecturas_de_un_sensor_por_cliente`

## N3 — API ↔ Base de datos

#### IT-DB-01 — Guarda la lectura con todas sus mediciones

- **Requisito:** RF-04 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** Bases vacías.
- **Pasos:** 1. POST de una lectura con 2 mediciones 2. GET de la lectura
- **Datos:** ubicación='Laboratorio'
- **Resultado esperado:** 1 lectura con 2 mediciones, seq, rssi y ubicación.
- **Automatización:** `tests/test_bd.py::test_it_db_01_guarda_lectura_con_todas_sus_mediciones`

#### IT-DB-02 — Marca única (sensor, seq) en PostgreSQL

- **Requisito:** RF-05 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** Lectura seq=1 guardada.
- **Pasos:** 1. Insertar otra marca con el mismo sensor y seq directamente en PostgreSQL
- **Datos:** —
- **Resultado esperado:** IntegrityError (la BD protege contra duplicados aunque falle la API).
- **Automatización:** `tests/test_bd.py::test_it_db_02_marca_unica_sensor_seq_a_nivel_de_bd`

#### IT-DB-03 — Mismo seq en sensores distintos es válido

- **Requisito:** RF-05 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST seq=1 para el sensor a y el sensor b
- **Datos:** —
- **Resultado esperado:** Ambos 201.
- **Automatización:** `tests/test_bd.py::test_it_db_03_mismo_seq_en_sensores_distintos_es_valido`

**IT-DB-04 — Nueva magnitud sin cambiar el esquema**

- **Requisito:** RF-07 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con co2_equiv además de T/H 2. GET
- **Datos:** co2_equiv=840
- **Resultado esperado:** Las 3 magnitudes se guardan y se consultan sin migrar nada.
- **Automatización:** `tests/test_bd.py::test_it_db_04_nueva_magnitud_sin_cambiar_esquema`

**IT-DB-05 — Borrado de cliente en cascada (sensores y marcas)**

- **Requisito:** RF-04 · **Prioridad:** Baja · **Tipo:** Automatizado (pytest)
- **Precondiciones:** Datos cargados.
- **Pasos:** 1. Borrar un cliente
- **Datos:** —
- **Resultado esperado:** Se eliminan sus sensores y marcas de seq.
- **Automatización:**
  `tests/test_bd.py::test_it_db_05_borrar_cliente_elimina_sensores_y_marcas_en_cascada`

**IT-DB-06 — seq negativo rechazado**

- **Requisito:** RF-04 · **Prioridad:** Baja · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con seq=-1
- **Datos:** —
- **Resultado esperado:** 422 antes de llegar a la BD.
- **Automatización:**
  `tests/test_bd.py::test_it_db_06_seq_negativo_rechazado_antes_de_llegar_a_la_bd`

**IT-DB-07 — Valores extremos sin pérdida**

- **Requisito:** RF-04 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST T=-40, H=100 2. GET
- **Datos:** —
- **Resultado esperado:** Valores idénticos.
- **Automatización:** `tests/test_bd.py::test_it_db_07_valores_extremos_se_almacenan_sin_perdida`

**IT-DB-08 — /health devuelve 503 si PostgreSQL no responde**

- **Requisito:** RNF-02 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. Iniciar la API con PostgreSQL inaccesible 2. GET /health
- **Datos:** —

- **Resultado esperado:** 503 y postgres='sin conexión'.
- **Automatización:** `tests/test_bd.py::test_it_db_08_health_503_si_postgresql_no_responde`

**IT-DB-09 — Zonas horarias normalizadas a UTC**

- **Requisito:** RNF-04 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con ts_dispositivo con offset -03:00 2. GET
- **Datos:** 2026-09-26T13:00:00-03:00
- **Resultado esperado:** Se almacena y devuelve como 16:00 UTC.
- **Automatización:** `tests/test_bd.py::test_it_db_09_ts_servidor_siempre_presente_y_utc`

**IT-DB-10 — Suite pytest contra PostgreSQL e InfluxDB reales (contenedores)**

- **Requisito:** RF-04 · **Prioridad:** Alta · **Tipo:** Semi-automático
- **Precondiciones:** Stack en ejecución; BD de pruebas `iot_test` creada; token de la API.
- **Pasos:** 1. Ejecutar pytest en un contenedor unido a la red nextguard_backend con TEST_DATABASE_URL=postgresql+psycopg://iot:@postgres:5432/iot_test, TEST_INFLUX_URL=http://influxdb:8086 y TEST_INFLUX_TOKEN=
- **Datos:** BD `iot_test`
- **Resultado esperado:** 52 pruebas aprobadas (49 con almacenamiento en memoria y 3 de integración con InfluxDB real).
- **Automatización:** `tests`

**IT-DB-11 — Contenedor influxdb o postgres detenido y recuperación sin reiniciar la API**

- **Requisito:** RNF-02 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** API y ESP32 funcionando.
- **Pasos:** 1. docker compose stop influxdb (5 min) y repetir con postgres 2. GET /health y observar el ESP32 3. docker compose start del servicio detenido 4. Observar
- **Datos:** —
- **Resultado esperado:** Durante la caída: /health 503 y POST con 503 (el ESP32 conserva la cola; la marca de seq se libera). Al volver, la API reconecta sola y se entrega la cola sin duplicados.

**IT-DB-12 — Almacén caído: 503 y el reintento posterior funciona**

- **Requisito:** RNF-02 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. Simular la caída del almacén 2. POST con seq=9 (503) 3. Restaurar y reenviar seq=9
- **Datos:** —
- **Resultado esperado:** 503 al caer; al volver, 201 y no «duplicado» (la marca se liberó).

- **Automatización:**
  `tests/test_bd.py::test_it_db_12_almacen_caido_503_y_el_reintento_posterior_funciona`

**IT-DB-13 — /health 503 si InfluxDB no responde**

- **Requisito:** RNF-02 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. Simular la caída del almacén 2. GET /health
- **Datos:** —
- **Resultado esperado:** 503 con influxdb='sin conexión' y postgres='ok'.
- **Automatización:** `tests/test_bd.py::test_it_db_13_health_503_si_el_almacen_no_responde`

**IT-DB-14 — Lectura sin seq no usa la marca de idempotencia**

- **Requisito:** RF-04 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST dos veces el formato de la clase (sin seq)
- **Datos:** —
- **Resultado esperado:** Ambos 201 y 2 lecturas guardadas.
- **Automatización:**
  `tests/test_bd.py::test_it_db_14_lectura_sin_seq_no_usa_marca_de_idempotencia`

**IT-DB-15 — InfluxDB real: guarda y devuelve la lectura**

- **Requisito:** RF-04 · **Prioridad:** Alta · **Tipo:** Semi-automático
- **Precondiciones:** TEST_INFLUX_URL y TEST_INFLUX_TOKEN definidos.
- **Pasos:** 1. POST y GET con un sensor aleatorio (se espera hasta 5 s por la visibilidad de InfluxDB)
- **Datos:** —
- **Resultado esperado:** La lectura vuelve con sus 2 mediciones, seq y rssi.
- **Automatización:**
  `tests/test_integracion_influx.py::test_it_db_15_influx_guarda_y_devuelve_la_lectura`

**IT-DB-16 — InfluxDB real: idempotencia por seq**

- **Requisito:** RF-05 · **Prioridad:** Alta · **Tipo:** Semi-automático
- **Precondiciones:** Ídem.
- **Pasos:** 1. POST 3 veces el mismo seq
- **Datos:** —
- **Resultado esperado:** Una sola lectura en InfluxDB.
- **Automatización:** `tests/test_integracion_influx.py::test_it_db_16_influx_idempotencia_por_seq`

**IT-DB-17 — InfluxDB real: formato de la clase**

- **Requisito:** RF-03 · **Prioridad:** Media · **Tipo:** Semi-automático

- **Precondiciones:** Ídem.
- **Pasos:** 1. POST con {sensor, cliente, tipo, valor, unidad}
- **Datos:** 19,5 °C
- **Resultado esperado:** El GET devuelve 19,5.
- **Automatización:** `tests/test_integracion_influx.py::test_it_db_17_influx_formato_de_la_clase`

#### IT-DB-18 — El token de Grafana es de solo lectura

- **Requisito:** RNF-03 · **Prioridad:** Alta · **Tipo:** Semi-automático
- **Precondiciones:** Stack en ejecución.
- **Pasos:** 1. Escribir un punto con el token de /tokens-grafana (curl a /api/v2/write dentro del contenedor influxdb) 2. Repetir con el token de la API 3. Comprobar que el datasource de Grafana funciona
- **Datos:** —
- **Resultado esperado:** Escribir con el token de Grafana → 403; con el de la API → 204; el datasource responde «1 buckets found».

## N4 — Extremo a extremo

#### IT-E2E-01 — Camino feliz: conteo exacto

- **Requisito:** RF-05 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** Entorno completo; ESP32 real.
- **Pasos:** 1. Dejar correr 10 min 2. Contar seq impresos en el monitor serie 3. Contar lecturas por sensor con GET /clientes//sensores//lecturas
- **Datos:** ≈ 60 lecturas
- **Resultado esperado:** Lecturas en InfluxDB = muestras enviadas: 0 faltantes y 0 duplicados.

#### IT-E2E-02 — Coherencia de valores serie ↔ BD

- **Requisito:** RF-04 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** Caso E2E-01 ejecutado.
- **Pasos:** 1. Comparar 10 lecturas al azar (seq, T, H)
- **Datos:** —
- **Resultado esperado:** Diferencia ≤ 0,1 (redondeo a 1 decimal).

#### IT-E2E-03 — Reinicio del ESP32: seq no se reutiliza

- **Requisito:** RF-05 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** ESP32 enviando.
- **Pasos:** 1. Anotar el último seq 2. Pulsar EN (reset) 3. Comparar el nuevo seq inicial
- **Datos:** —

- **Resultado esperado:** El nuevo seq es mayor (salto de hasta 20 por diseño); no hay lecturas sobrescritas ni duplicadas.

**IT-E2E-04 — Dos dispositivos simultáneos**

- **Requisito:** RF-05 · **Prioridad:** Media · **Tipo:** Semi-automático
- **Precondiciones:** ESP32 real activo.
- **Pasos:** 1. python simulador_sensor.py --devices 1 --count 60 --interval 10 --verify (mientras corre el ESP32)
- **Datos:** —
- **Resultado esperado:** Cada dispositivo conserva su propia secuencia; sin interferencia.
- **Automatización:** `scripts/simulador_sensor.py`

**IT-E2E-05 — Trazabilidad temporal**

- **Requisito:** RNF-04 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** NTP funcionando.
- **Pasos:** 1. Consultar 20 lecturas 2. Calcular |ts_servidor − ts_dispositivo|
- **Datos:** —
- **Resultado esperado:** Diferencia < 60 s (en operación normal).

**IT-E2E-06 — Consulta por rango de fechas**

- **Requisito:** RF-06 · **Prioridad:** Media · **Tipo:** Semi-automático
- **Precondiciones:** ≥ 30 min de datos.
- **Pasos:** 1. GET ?desde=…&hasta=… con un rango de 5 min
- **Datos:** —
- **Resultado esperado:** Solo lecturas dentro del rango, ordenadas de la más reciente.

**IT-E2E-07 — Nueva magnitud extremo a extremo**

- **Requisito:** RF-07 · **Prioridad:** Media · **Tipo:** Semi-automático
- **Precondiciones:** API en EC2.
- **Pasos:** 1. python simulador_sensor.py --with-pm --with-mq --devices 1 --count 10 --verify
- **Datos:** pm2_5, co2_equiv
- **Resultado esperado:** Se guardan y se consultan sin migrar el esquema.
- **Automatización:** `scripts/simulador_sensor.py`

**IT-E2E-08 — ESP32 real con DHT11 + MQ135 hasta el dashboard de Grafana**

- **Requisito:** RF-08 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** Sensores cableados (MQ135 con divisor); portal configurado con el cliente y el sensor; stack en la EC2.
- **Pasos:** 1. Dejar correr 10 min 2. Abrir el dashboard, elegir cliente y sensor 3. Comparar el último valor de cada panel con el monitor serie

- **Datos:** —
- **Resultado esperado:** Los paneles de temperatura, humedad y calidad del aire muestran los valores del monitor serie (±0,1 °C / ±0,5 %HR) sin huecos.

## N5 — No funcionales

#### IT-NFR-01 — Latencia local: 200 envíos, p95 < 0,5 s

- **Requisito:** RNF-01 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** Almacenamiento en memoria y SQLite locales.
- **Pasos:** 1. Ejecutar la prueba de carga ligera
- **Datos:** 200 envíos
- **Resultado esperado:** p95 < 0,5 s.
- **Automatización:**
  `tests/test_seguridad_y_carga.py::test_it_nfr_01_200_envios_secuenciales_p95_bajo_umbral`

#### IT-NFR-02 — Reintentos masivos sin duplicados

- **Requisito:** RF-05 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. Enviar 20 seq × 5 veces
- **Datos:** —
- **Resultado esperado:** Exactamente 20 filas.
- **Automatización:**
  `tests/test_seguridad_y_carga.py::test_it_nfr_02_reintentos_masivos_no_generan_duplicados`

#### IT-NFR-03 — Latencia real hacia AWS (p95 < 2 s)

- **Requisito:** RNF-01 · **Prioridad:** Alta · **Tipo:** Semi-automático
- **Precondiciones:** API en EC2; simulador ejecutado desde fuera de AWS.
- **Pasos:** 1. python simulador_sensor.py --url http://:8000 --devices 3 --count 100 --interval 1 --parallel --verify
- **Datos:** —
- **Resultado esperado:** p95 < 2 s; 0 errores de red; verificación OK.
- **Automatización:** `scripts/simulador_sensor.py`

#### IT-NFR-04 — Carga: 20 dispositivos × 10 min

- **Requisito:** RNF-01 · **Prioridad:** Media · **Tipo:** Semi-automático
- **Precondiciones:** Entorno AWS completo.
- **Pasos:** 1. python simulador_sensor.py --devices 20 --count 600 --interval 1 --parallel --verify 2. Observar CPU/memoria de la EC2 (top) y conexiones de PostgreSQL (docker stats)
- **Datos:** 20 dispositivos, 1 lectura/s

- **Resultado esperado:** 0 respuestas 5xx; verificación OK; CPU EC2 < 80 %.
- **Automatización:** `scripts/simulador_sensor.py`

**IT-NFR-05 — Soak de 3 h con ESP32 real**

- **Requisito:** RNF-02 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** Alimentación estable; sesión del laboratorio con ≥ 3,5 h restantes.
- **Pasos:** 1. Dejar funcionando 3 h 2. Cada 30 min revisar el monitor serie 3. Al final: comparar enviadas vs BD y buscar saltos de seq ≥ 20 (reinicios)
- **Datos:** ≈ 1080 lecturas
- **Resultado esperado:** Entrega ≥ 99 %, 0 duplicados, 0 reinicios inesperados.

**IT-NFR-06 — Corte de energía del ESP32**

- **Requisito:** RNF-02 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** ESP32 enviando.
- **Pasos:** 1. Desconectar la alimentación 1 min 2. Reconectar
- **Datos:** —
- **Resultado esperado:** Lo ya guardado permanece íntegro; se reanuda solo. Se documenta la pérdida máxima (cola en RAM ≤ 60 muestras).

**IT-NFR-07 — Reinicio de la EC2**

- **Requisito:** RNF-02 · **Prioridad:** Alta · **Tipo:** Manual
- **Precondiciones:** Elastic IP asignada.
- **Pasos:** 1. Reiniciar la instancia 2. Esperar el arranque 3. Observar el ESP32
- **Datos:** —
- **Resultado esperado:** Los contenedores arrancan solos (restart: unless-stopped) y el ESP32 reanuda el envío. Sin Elastic IP la IP cambia: se documenta como riesgo R-01.

**IT-NFR-08 — Cola llena tras 15 min sin red**

- **Requisito:** RF-05 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** Firmware con COLA_MAX=60 e INTERVALO=10 s.
- **Pasos:** 1. Apagar el router 15 min 2. Encenderlo 3. Comparar con la BD
- **Datos:** —
- **Resultado esperado:** Se conservan las últimas 60 muestras, las más antiguas se descartan y el contador `perdidas` coincide con las faltantes en la BD.

**IT-NFR-09 — Crecimiento de InfluxDB y PostgreSQL**

- **Requisito:** RNF-02 · **Prioridad:** Baja · **Tipo:** Semi-automático
- **Precondiciones:** ≥ 10 000 lecturas cargadas con el simulador.
- **Pasos:** 1. docker compose exec influxdb du -sh /var/lib/influxdb2 2. docker compose exec postgres psql -U iot -d iot -c 'SELECT

pg_size_pretty(pg_database_size(current_database()))'

- **Datos:** —
- **Resultado esperado:** Se documenta el tamaño por 1000 lecturas y la proyección a 30 días para 10 dispositivos.

## N6 — Seguridad básica

#### IT-SEC-01 — Inyección SQL en sensor

- **Requisito:** RNF-03 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con sensor="x'; DROP TABLE sensores;--"
- **Datos:** —
- **Resultado esperado:** 422; las tablas siguen intactas.
- **Automatización:**
  `tests/test_seguridad_y_carga.py::test_it_sec_01_inyeccion_sql_en_sensor_rechazada`

#### IT-SEC-02 — Inyección SQL en texto libre

- **Requisito:** RNF-03 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con ubicacion="'); DROP TABLE sensores;--"
- **Datos:** —
- **Resultado esperado:** Se guarda literalmente; las tablas siguen intactas.
- **Automatización:**
  `tests/test_seguridad_y_carga.py::test_it_sec_02_inyeccion_sql_en_texto_libre_se_guarda_literal`

#### IT-SEC-03 — Inyección SQL y Flux en filtros GET

- **Requisito:** RNF-03 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. GET ?sensor=a' OR '1'='1 2. GET ?cliente=a") |> drop() //
- **Datos:** —
- **Resultado esperado:** 422.
- **Automatización:**
  `tests/test_seguridad_y_carga.py::test_it_sec_03_inyeccion_en_filtro_get_rechazada`

#### IT-SEC-04 — Tipo de medición con caracteres no permitidos

- **Requisito:** RNF-03 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST con tipo='temp; DROP'
- **Datos:** —
- **Resultado esperado:** 422.

- **Automatización:**
  `tests/test_seguridad_y_carga.py::test_it_sec_04_tipo_con_caracteres_no_permitidos_422`

**IT-SEC-05 — Sin claves configuradas se rechaza todo**

- **Requisito:** RNF-03 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. Iniciar la API con API_KEYS vacío 2. POST con cualquier clave
- **Datos:** —
- **Resultado esperado:** 401 (falla cerrada).
- **Automatización:**
  `tests/test_seguridad_y_carga.py::test_it_sec_05_sin_claves_configuradas_se_rechaza_todo`

**IT-SEC-06 — InfluxDB y PostgreSQL no accesibles desde Internet**

- **Requisito:** RNF-03 · **Prioridad:** Alta · **Tipo:** Semi-automático
- **Precondiciones:** Compose sin `ports` en influxdb y postgres; SG sin reglas para 8086 ni 5432.
- **Pasos:** 1. Desde tu PC (fuera de AWS): EC2_HOST= ./scripts/smoke_env.sh --externo
- **Datos:** —
- **Resultado esperado:** Los puertos 8086 y 5432 no responden desde fuera.
- **Automatización:** `scripts/smoke_env.sh --externo`

**IT-SEC-07 — Sin secretos en el código**

- **Requisito:** RNF-03 · **Prioridad:** Alta · **Tipo:** Semi-automático
- **Precondiciones:** —
- **Pasos:** 1. grep -rniE "(password|api_key|secret)\s*[=:]" . --include=*.ino --include*=.py --include=*.h --include*=.md 2. Verificar que config.h y .env están en .gitignore
- **Datos:** —
- **Resultado esperado:** Solo aparecen valores de ejemplo (`CAMBIAR`, `TU_CLAVE`); ningún secreto real.

**IT-SEC-08 — La API key no aparece en los logs**

- **Requisito:** RNF-03 · **Prioridad:** Media · **Tipo:** Manual
- **Precondiciones:** Stack en ejecución.
- **Pasos:** 1. Enviar lecturas con clave válida y errónea 2. docker compose logs api | grep -i 3. docker compose logs grafana influxdb | grep -i
- **Datos:** —
- **Resultado esperado:** Sin coincidencias.

**IT-SEC-09 — Tráfico en claro vs HTTPS (fase 2)**

- **Requisito:** RNF-03 · **Prioridad:** Baja · **Tipo:** Manual

- **Precondiciones:** Captura con Wireshark en la red del ESP32.
- **Pasos:** 1. Capturar con HTTP 2. Repetir con USE_HTTPS=1
- **Datos:** —
- **Resultado esperado:** Con HTTP se ve la X-API-Key en claro (riesgo aceptado en laboratorio, documentado); con HTTPS no.

**IT-SEC-10 — Los endpoints de clientes exigen API key**

- **Requisito:** RNF-03 · **Prioridad:** Alta · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST /clientes, GET /clientes y GET /clientes//sensores sin X-API-Key
- **Datos:** —
- **Resultado esperado:** 401 en todos.
- **Automatización:**
  `tests/test_seguridad_y_carga.py::test_it_sec_10_endpoints_de_clientes_exigen_api_key`

**IT-SEC-11 — Identificador de cliente o correo inválido → 422**

- **Requisito:** RNF-03 · **Prioridad:** Media · **Tipo:** Automatizado (pytest)
- **Precondiciones:** —
- **Pasos:** 1. POST /clientes con id 'a b' 2. POST /clientes con email inválido
- **Datos:** —
- **Resultado esperado:** 422.
- **Automatización:**
  `tests/test_seguridad_y_carga.py::test_it_sec_11_identificador_de_cliente_invalido_422`